

Formally Analyzing Microarchitecture Using Computation Tree Logic (CTL) to Identify Security Vulnerabilities

Evan Wong

University of California, Berkeley
ezw@berkeley.edu

Abstract—This project was done for the final project of the University of California, Berkeley’s class EE 144: Fundamental Algorithms for Systems Modeling, Analysis, and Optimization, taught by Professor Pierluigi Nuzzo and Professor Alberto Sangiovanni-Vincentelli.

Index Terms—Computation Tree Logic, formal verification, computer architecture, branch prediction, RISC-V

I. INTRODUCTION

Many modern processors include features like speculative execution and out-of-order execution. These boost performance, but they also unintentionally leak information through microarchitectural states demonstrated in attacks like Spectre [6] (speculative execution + branch prediction) and Meltdown [5] (accessing privileged memory through OoO execution). Detecting them with formal tools (rather than manually building proof-of-concept attacks) could be a way to more concretely prevent future attacks using Computation Tree Logic (CTL) to formalize these desired properties when designing microarchitecture.

II. METHODOLOGY

This initial idea for the project aimed to detect violations of information flow security using Linear Temporal Logic (LTL) for property specification and SAT-based model checking for analysis. The goal was to fully formally specify the properties of a Minimal RTL-Level or FSM-Level Model BOOM Chip from Berkeley’s open-sourced Chipyard interface, where the project would specifically select very minimal configurations to limit the scale of the application initially.

The idea was to start with a minimum design of a branch predictor and speculative fetch unit, and what could be a tiny ISA with just loads, stores, and branches, because caches are a much greater challenge to design in terms of complexity to model. Using Spike and possibly EDA formal verification tools such as JasperGold and/or SymbiYosys, architectural states/registers can be checked after the operation to verify the intended behavior based on the CTL.

What ended up happening when starting the project was a realization that configurability within the microarchitectures specified at a higher level, such as in C or C++, was not ideal for working with specific caching requirements and tinkering with out-of-order complexity and speculation, so the project was limited down to focusing simply on RISC-V

branching instructions in the context of branch prediction as a microarchitectural optimization. This decision was primarily made due to my lack of skill with any hardware description language like Verilog, where I could have (not easily) written a custom datapath to isolate components I needed, such as an ALU. Additionally, further research was done into the limitations of LTL compared to Computation Tree Logic (CTL) [1], and LTL was deemed unsuitable to model a majority of the microarchitectures proposed due to the idea of multiple possibilities along a path. Thus, the approach was shifted to use CTL in the context of branch prediction in standard in-order pipelines.

III. RELATED WORKS

K. Cheang, C. Rasmussen, S. A. Seshia, and P. Subramanyan, “A Formal Approach to Secure Speculation,” establishes a framework for exposing transient execution vulnerabilities like Spectre for cache/memory traces, branch predictor state, and architectural state by establishing TPOD (Trace Property-Dependent Observational Determinism), although it is extremely mathematically intense to read. The approach in this paper does not go anywhere near as mathematically rigorous as this one, but it attempts to provide a simple proof of concept for similar frameworks. [4]

M. R. Fadiheh et al., “An Exhaustive Approach to Detecting Transient Execution Side Channels in RTL Designs of Processors,” presents UPEC (Unique Program Execution Checking), a formal verification methodology for detecting transient execution side channels in the RTL designs of processors. The framework was successfully deployed on both RocketChip and BOOM processors, so it essentially covers a lot of topics in the original proposal. However, this framework doesn’t use any explicit sort of logic formalization, such as CTL. [3]

With these studies in mind, the main goal of this project was to expose myself to formal toolchains such as JasperGold and analyze microarchitectures from a more formal perspective. I’d never personally used the Spike simulator before to analyze traces, and connecting the traces and architectural states to a formal tool like JasperGold was completely foreign to me.

IV. IMPLEMENTATION AND CHALLENGES

Starting with some large logistical challenges, the first step to working with Chipyard was to either run Chipyard

```

class Project144Config extends Config {
  new With1TinyCore ++
  new WithNoMemPort ++
  new WithNMemoryChannels(0) ++
  new WithNBanks(0) ++
  new WithIncoherentBusTopology ++
  new BaseConfig
}

```

Fig. 1. Config File

through SSHing into a virtual machine or install Chipyard on a local virtual machine or Linux environment. Due to a lack of personal access to machines with Chipyard and its tooling already installed, I had to set up my own Linux environment on my devices along with Chipyard. Chipyard is too large, so you have to download it manually onto your local machine while keeping it separate from repositories. GitHub repositories were my main method of transferring and saving specific configuration files that I needed, and the one I specifically used for this project is publicly accessible at this link: <https://github.com/evanzwong/ArchitectureFormalSpec>

To start the SoC building process, you must specify your specific SoC configuration in “chipyard/generators/rocket-chip/src/main/scala/system/Configs.scala”. For the sake of this project, I added a configuration class called “Project144Config”. To make the most minimal SoC configuration possible for the branch prediction datapath, I copied a few standard configuration settings, which looked like this:

The 1TinyCore parameter forces there to be a single in-order RISC-V processor (a RocketCore), and some of the other ones just minimize extra features in a RocketCore to be as barebones as possible as per our expected specifications for a test run (the lack of memory ports was to test architectural registers as proof of concept). Additionally, we had to modify a configuration inside the RocketChip itself, so we did a very similar configuration for that in “chipyard/generators/chipyard/src/main/scala/config /RocketConfigs.scala”

Something new I learned from having to edit these configurations later was that despite a lack of memory hierarchy and a seemingly removal of all caches, it is actually impossible to remove the L1 I\$ in a RocketCore, so I was unable to achieve my original goal of removing caching for this project. If I wrote my own custom processor in HDL, this would be easily achievable.

Following these configurations, I formulated a RISC-V test to toss into a RocketCore to test. The actual test file is in the GitHub repository, but at a high level, the goal was to train the basic bimodal branch

predictor included in the BOOM processor at “chipyard/generators/boom/src/main/resources/csrc/predictor_sw.cc” to max out its bits to equal 3 (so 11 in binary) such that when it finally escapes its loop, it will still remain “slightly inclined to branch” (10) on the next branch. Using this test, we can place a branch that isn’t meant to be taken, followed by the correct architectural state, but have the branch lead to somewhere harmful. In the cycles where it doesn’t flush the pipeline, the mispredicted branch will continue to execute harmful instructions, which could be favorable to an attacker (which we obviously didn’t implement here). However, I came to another realization that we couldn’t directly transfer this C branch predictor file to the RocketCore core configuration, so we couldn’t customize the RocketCore branch predictor beyond its base configuration (which was fine in this case).

The final step for actually running all my Chipyard configurations was to simulate the processor using Spike, the RISC-V golden model, which can dump traces. I actually couldn’t figure out how to use this properly on a custom RISC-V test in Chipyard through watching videos and reading articles on how to interface it, but if I were able to figure it out, it would have been relatively straightforward to just take the file dumps and also a Verilog dump to throw into JasperGold. However, I was not able to get that far myself, and JasperGold was also difficult to use in asserting so many architectural properties in the virtual machine that wasn’t even mine was proving to be a challenge, so this is where my progress ended up in the end.

V. SUMMARY AND CONCLUSION

While this project didn’t end up creating a framework as I’d hoped it would due to a technical misunderstanding of the ease of use of tools, this project was a great introduction to interfacing many different tools within the context of EDA and a formal approach to the design of microarchitectures. Through the trials and tribulations of this few-week project, I now truly understand the importance of an efficient and easy-to-use toolchain for EDA. If I were to continue working on this project, I would definitely have an easier time interfacing my toolchain if everything were on the same device or at least easily accessible to me, which also shows how much room for improvement the open-source market has. Given I also had time to create a custom minimalist processor in Verilog, this project would have also been easier to interface with Verilog-only tools. Despite the lack of hard results, there are many more avenues to explore beyond the scope of this class project:

- Actual analysis of security regarding memory, rather than trying to use the Spike simulator to show architectural traces
- Analysis of BOOM, the Chipyard Out-of-Order processor, to analyze speculative execution and more costly branch mispredictions
- For barebones architectural state analysis, write a custom Verilog datapath to not include any caching (impossible in current Chipyard configurations)

VI. CONTRIBUTIONS

All material in the repository and explored in this paper were either from Evan or adapted from references. Access to JasperGold (although not explored in this paper extensively) was credited to graduate students and administration at the UC Berkeley SLICE lab.

VII. PROJECT REPOSITORY

All resources and scripts are available at:
<https://github.com/evanzwong/ArchitectureFormalSpec>

REFERENCES

- [1] A. Sangiovanni Vincentelli and P. Nuzzo, "Lecture 11: Formal Specification: Temporal Logic," EE 144/244 – Spring 2025, University of California, Berkeley, Mar. 31, 2025.
- [2] K. Asanović *et al.*, "The Rocket Chip Generator," EECS Technical Report No. UCB/EECS-2016-17, UC Berkeley, Apr. 2016. [Online]. Available: <https://github.com/ucb-bar/rocket-chip>
- [3] M. R. Fadiheh, A. Gens, J. Park, M. Schölzel, A. Kriegel, and A. Beyer, "An Exhaustive Approach to Detecting Transient Execution Side Channels in RTL Designs of Processors," *IEEE Transactions on Computers*, vol. 71, no. 8, pp. 1766–1782, 2022, doi: 10.1109/TC.2022.3152666.
- [4] K. Cheang, C. Rasmussen, S. A. Seshia, and P. Subramanyan, "A Formal Approach to Secure Speculation," *arXiv preprint arXiv:1903.00446*, 2019.
- [5] M. Lipp *et al.*, "Meltdown: Reading Kernel Memory from User Space," in *Proc. 27th USENIX Security Symposium*, 2018. [Online]. Available: <https://meltdownattack.com/meltdown.pdf>
- [6] P. Kocher *et al.*, "Spectre Attacks: Exploiting Speculative Execution," in *Proc. 40th IEEE Symposium on Security and Privacy (S&P)*, 2019. [Online]. Available: <https://spectreattack.com/spectre.pdf>